Research: Using Gemma 4 for Real-Time ASL Translation

Overview

This document covers what Google's Gemma 4 model is, why it could be a fit for our ASL translation device, and how it would slot into our existing pipeline. Gemma 4 was released by Google DeepMind in April 2026 as their most capable open model family for on-device use, with strong support for edge hardware, including the NVIDIA Jetson series, which we are already using.

What is Gemma 4

Gemma 4 is an open-weight model family from Google, built on the same research as Gemini 3. It comes in four sizes that target different deployment scenarios:

- **E2B (Effective 2B)** for very small edge devices like phones and Raspberry Pi
- **E4B (Effective 4B)** is also edge-focused, slightly more capable
- **26B A4B** (Mixture of Experts) for consumer GPUs and workstations
- **31B Dense** for the largest workstation deployments

The "E" sizes are the ones relevant to us since we are deploying on the Jetson.

Key capabilities relevant to our project:

- **Multimodal input** with native support for text, images, and audio (audio supported on small models)
- **Long context windows** up to 128K tokens on the small models
- **Native function calling and agentic workflows**
- **Apache 2.0 license**, so commercial and academic use are both fine
- **Day one support** for Ollama, vLLM, Hugging Face Transformers, llama.cpp, LiteRT-LM, and NVIDIA NIM

How Gemma 4 Could Fit Our Pipeline

Our current pipeline is MediaPipe (hand landmark extraction) plus an LSTM trained on WLASL for sign classification, optimized with TensorRT, and deployed on the Jetson. There are a few ways Gemma 4 could improve this:

Option A: Post-Processing Layer

Keep MediaPipe + LSTM for raw sign recognition, then use Gemma 4 as a natural language post-processor that turns a sequence of recognized signs into fluent, grammatical English. ASL grammar is different from English, so a small LLM can help bridge that gap.

Example: LSTM outputs ["YESTERDAY", "I", "GO", "STORE"] and Gemma 4 turns it into "Yesterday I went to the store."

Option B: Direct Video Understanding

Feed video frames directly to Gemma 4 using its multimodal vision capability and ask it to interpret the signing. This would replace the MediaPipe + LSTM pipeline entirely, but would likely be slower and less accurate on isolated signs than a model trained specifically for that task.

Option C: Hybrid System

Use the LSTM for primary recognition and call Gemma 4 only when the LSTM has low confidence, using its broader reasoning to help disambiguate.

Pros and Cons for Our Project

Pros

- Runs on Jetson hardware out of the box (Jetson Orin Nano explicitly supported)
- Open weights with permissive license, no API costs
- Strong multilingual support if we ever expand beyond ASL
- Active community and good tooling support
- Can run fully offline, which preserves the privacy story for our device

Cons

- Adds another model to the pipeline, which increases latency and memory usage
- A general-purpose LLM is not optimized for ASL specifically, so a custom LSTM is likely still more accurate for raw sign recognition
- More complexity to maintain and deploy
- Need to fine-tune or carefully prompt for ASL specific tasks

Initial Recommendation

The most realistic and useful integration for our project is **Option A: post-processing layer**. We keep our existing MediaPipe + LSTM pipeline (which is purpose-built for sign recognition

and faster) and add Gemma 4 E2B or E4B as a final step that converts the raw sign sequence into natural English sentences.

This gives us the biggest user-facing improvement (output reads like real English rather than a list of signs) for the least disruption to what we have already built. We can start with the E2B model since it has the smallest memory footprint and see if the accuracy is good enough before moving to E4B.

Next Steps

- Test Gemma 4 E2B locally via Ollama or LiteRT-LM to see baseline performance
- Prototype the sign sequence to English conversion with a few sample inputs
- Measure added latency on the Jetson hardware
- Decide whether E2B is sufficient or if we need to move up to E4B